



Rapport projet d'Algorithmie des graphes

Résolution du problème du voyageur de commerce

Tristan GROULT, Louis GUERAIN, Kibandou BIGNANG

2 janvier 2026

Sommaire

1 Notice d'utilisation **3**

2 Algorithme **3**

2.1 Point le Plus Proche (PPP) 3
2.2 Point le Plus Proche Optimisé (OptPPP) 7
2.3 Parcour en largeur d'un ACM à partir de Prim (OptPrim) 9
2.4 OptPrim + Décroisé (OptOptPrim) 17
2.5 Évaluation et séparation progressive ("branch and bound") (HDS) pas fait . . 17

1 Notice d'utilisation

Ce projet dépend des bibliothèques `numpy` et d'autres modules standards. Il dépend aussi de `flask` pour l'application web.

```
pip install numpy # (tout)
pip install flask # (web app)
```

Listing 1 – Installation des dépendances

Ou plus simplement :

```
pip install requirement.txt
```

2 Algorithme

2.1 Point le Plus Proche (PPP)

Algorithme : Procédure PPP (Point le Plus Proche)
Description : Construit un cycle hamiltonien en insérant itérativement le point le plus proche du cycle actuel.
Vars :
- c : Liste d'entiers (Cycle hamiltonien)
- l : Réel (Longueur du cycle)
- Q_i, Q_j : Entiers (Indices des points sélectionnés)

Entrées :
- L : Liste de points $P_1, \ldots, P_n$
- Q : Un point de départ choisi dans L
- D : La matrice des distances $n \times n$

Sorties :
- c : Un cycle hamiltonien
- l : La longueur du cycle

Début :

1. $c \leftarrow$ cycle trivial composé du seul point Q
2. $l \leftarrow 0$
3. Retirer Q de L
4. Tant que L n'est pas vide faire :
5. $\quad (Q_i, Q_j) \leftarrow \texttt{TrouverCoupleMin}(L, c, D)$
6. $\quad (c, l) \leftarrow \texttt{InsérerDansCycle}(c, Q_i, Q_j, D, l)$
7. $\quad$ Retirer Q_i de L
8. Retourner c, l

Algorithme : InsérerDansCycle
Description : Insère Q_i dans c soit entre Q_j et son successeur, soit entre son prédécesseur et Q_j, selon le coût minimal, et met à jour la longueur du cycle.
Vars :
— Q_{next}, Q_{prev} : Entiers (Indices des voisins de Q_j)
— $\text{coût}_{\text{droite}}, \text{coût}_{\text{gauche}}$: Réels (Coûts d'insertion)
Entrées :
— c : Cycle partiel
— Q_i : Point à insérer
— Q_j : Point d'ancrage dans c
— D : Matrice des distances
— l : Longueur actuelle du cycle
Sorties : c (cycle mis à jour), l (nouvelle longueur)
Début :

1. Soit Q_{next} le successeur de Q_j dans c
2. Soit Q_{prev} le prédécesseur de Q_j dans c
3. $\text{coût}_{\text{droite}} \leftarrow D[Q_j, Q_i] + D[Q_i, Q_{next}] - D[Q_j, Q_{next}]$
4. $\text{coût}_{\text{gauche}} \leftarrow D[Q_{prev}, Q_i] + D[Q_i, Q_j] - D[Q_{prev}, Q_j]$
5. Si $\text{coût}_{\text{droite}} \leq \text{coût}_{\text{gauche}}$ alors :
6. Insérer Q_i entre Q_j et Q_{next} dans c
7. $l \leftarrow l + \text{coût}_{\text{droite}}$
8. Sinon :
9. Insérer Q_i entre Q_{prev} et Q_j dans c
10. $l \leftarrow l + \text{coût}_{\text{gauche}}$
11. Retourner c, l

Algorithme : TrouverCoupleMin
Description : Trouve le couple (Q_i, Q_j) tel que $Q_i \in L$, $Q_j \in c$ et la distance $D[Q_i, Q_j]$ est minimale.
Vars :
— d_{min} : Réel (Distance minimale trouvée)
— u, v : Entiers (Indices de parcours)
Entrées :
— L : Liste des points non encore visités
— c : Le cycle partiel actuel
— D : La matrice des distances $n \times n$
Sorties : Q_i (point sélectionné dans L), Q_j (point d'ancrage dans c)
Début :

1. $d_{min} \leftarrow \infty$
2. $Q_i \leftarrow$ vide
3. $Q_j \leftarrow$ vide
4. Pour chaque point u appartenant à L faire :
5. Pour chaque point v appartenant à c faire :
6. Si $D[u, v] < d_{min}$ alors :
7. $d_{min} \leftarrow D[u, v]$
8. $Q_i \leftarrow u$
9. $Q_j \leftarrow v$
10. Retourner Q_i, Q_j

Implémentation Python :

```python
def trouver_couple_min(L, c, D):
    """
    Trouve le couple (Q_i, Q_j) tel que Q_i dans L, Q_j dans c et D[Q_i,
    Q_j] est minimale.
    Args:
        L (list): Liste des indices des points non encore visités
        c (list): Liste des indices du cycle partiel actuel
        D (np.ndarray): Matrice des distances n x n
    Returns:
        tuple: (Q_i, Q_j) indices
    """
    d_min = float('inf')
    Q_i = None
    Q_j = None
    for u in L:
        for v in c:
            if D[u, v] < d_min:
                d_min = D[u, v]
                Q_i = u
                Q_j = v
    return Q_i, Q_j

def inserer_dans_cycle(c, Q_i, Q_j, D, l):
    """
    Insère Q_i dans c entre Q_j et son point adjacent, met à jour la
```

```
    longueur du cycle.
    Args:
        c (list): Liste des indices du cycle partiel
        Q_i (int): Indice du point à insérer
        Q_j (int): Indice du point d'ancrage dans c
        D (np.ndarray): Matrice des distances
        l (float): Longueur actuelle du cycle
    Returns:
        tuple: (c, l) cycle mis à jour et nouvelle longueur
    """
    n = len(c)
    idx_j = c.index(Q_j)

    # Insertion entre Q_j et le suivant (j+1)
    idx_next = (idx_j + 1) % n
    Q_next = c[idx_next]
    cout_droite = D[Q_j, Q_i] + D[Q_i, Q_next] - D[Q_j, Q_next]

    # Insertion entre le précédent (j-1) et Q_j
    idx_prev = (idx_j - 1) % n
    Q_prev = c[idx_prev]
    cout_gauche = D[Q_prev, Q_i] + D[Q_i, Q_j] - D[Q_prev, Q_j]

    # Choix du meilleur
    if cout_droite <= cout_gauche:
        c.insert(idx_j + 1, Q_i)
        l += cout_droite
    else:
        c.insert(idx_j, Q_i)
        l += cout_gauche

    return c, l

def ppp(D, Q=0):
    """
    Procédure PPP : construit un cycle hamiltonien en insérant ité
    rativement le point le plus proche du cycle actuel.
    Args:
        D (np.ndarray): Matrice des distances.
        Q (int, optional): Indice du point de départ. Si None, prend 0.
    Returns:
        tuple: (c, l) cycle hamiltonien (liste d'indices), longueur du
    cycle
    """
    n = len(D)
    if n == 0:
        return [], 0.0
    L = list(range(n))
    c = [Q]
    l = 0.0
    L.remove(Q)
    while L != []:
        Q_i, Q_j = trouver_couple_min(L, c, D)
        c, l = inserer_dans_cycle(c, Q_i, Q_j, D, l)
        L.remove(Q_i)
    return c, l
```

Listing 2 – Implémentation de PPP

2.2 Point le Plus Proche Optimisé (OptPPP)

Algorithme : Procédure Décroise
Description : Améliore un cycle en décroisant les arêtes tant que cela réduit la distance totale.
Vars :
- n : Entier (Nombre de points)
- amélioration : Booléen (Indicateur de modification)
- i, j : Entiers (Indices de boucle)
- u, v, x, y : Entiers (Indices des points des arêtes)
- $d_{actuelle}, d_{nouvelle}$: Réels (Distances comparées)
- p, q : Entiers (Indices pour l'inversion)

Entrées :
- c : Cycle hamiltonien initial
- D : Matrice des distances

Sorties : c (cycle optimisé), l (longueur)
Début :

1. $n \leftarrow$ taille(c)
2. amélioration $\leftarrow$ Vrai
3. Tant que amélioration faire :
4. amélioration $\leftarrow$ Faux
5. Pour i allant de 1 à $n-2$ faire :
6. Pour j allant de $i+2$ à n faire :
7. $u \leftarrow c[i]$
8. $v \leftarrow c[i+1]$
9. $x \leftarrow c[j]$
10. Si $j = n$ alors :
11. $y \leftarrow c[1]$
12. Sinon :
13. $y \leftarrow c[j+1]$
14. $d_{actuelle} \leftarrow D[u, v] + D[x, y]$
15. $d_{nouvelle} \leftarrow D[u, x] + D[v, y]$
16. Si $d_{nouvelle} < d_{actuelle}$ alors :
17. $c[i+1 \dots j] \leftarrow$ inverser$(c[i+1 \dots j])$
18. amélioration $\leftarrow$ Vrai
19. $l \leftarrow$ CalculerLongueur(c, D)
20. Retourner c, l

Algorithme : Procédure OptPPP
Description : Wrapper pour l'algorithme de décroisement.
Vars : Aucune
Entrées :
— c : Cycle hamiltonien initial
— D : Matrice des distances
Sorties : c (cycle optimisé), l (longueur)
Début :
1. Retourner `Décroise`(c, D)

Implémentation Python :

```python
def decroise(c, D):
    """
    Améliore le cycle en décroisant les arêtes.
    Args:
        c (list): Cycle hamiltonien (liste d'indices)
        D (np.ndarray): Matrice des distances
    Returns:
        tuple: (c, l) cycle optimisé et sa nouvelle longueur
    """
    n = len(c)
    amelioration = True
    while amelioration:
        amelioration = False
        for i in range(n - 2):
            for j in range(i + 2, n):
                u = c[i]
                v = c[i + 1]
                x = c[j]
                y = c[(j + 1) % n]
                d_act = D[u, v] + D[x, y]
                d_nouv = D[u, x] + D[v, y]
                if d_nouv < d_act:
                    # Inversion de la sous-séquence entre i+1 et j
                    c[i+1:j+1] = c[i+1:j+1][::-1]
                    amelioration = True

    # Recalcul de la longueur totale du cycle optimisé
    l = 0.0
    for k in range(n):
        l += D[c[k], c[(k + 1) % n]]

    return c, l

def optppp(D, r0=0):
    """
    Procédure OptPPP : améliore le cycle fourni par PPP en décroisant
   les arêtes.
    Args:
        D (np.ndarray): Matrice des distances
        r0 (int, optional): Sommet de départ. Par défaut 0.
    Returns:
        tuple: (c, l) cycle optimisé et sa nouvelle longueur
    """
    c, _ = ppp(D, r0)
```

```
    return decroise(c, D)
```

Listing 3 – Implémentation de OptPPP

2.3 Parcour en largeur d'un ACM à partir de Prim (OptPrim)

Structure de données : GraphePrim
Type GraphePrim = **Enregistrement**
cle : Tableau[1..n] de Réel ;
pred : Tableau[1..n] d'Entier ;
NoeudTas : Tableau[1..n] d'Entier
Tas : Tableau[1..n] d'Entier
1 **Fin** ;

Algorithme : Prim (Optimisé)
Description : Construit un arbre couvrant de poids minimum (MST) en utilisant un tas binaire pour sélectionner efficacement le prochain sommet le plus proche.
Vars :
— $GraphePrim$: Structure GraphePrim (Données du graphe et du tas)
— $pres$: Tableau de booléens (Indique si un sommet est dans le tas)
— u, v : Entiers (Indices de sommets)
Entrées :
— D : Matrice des distances $n \times n$
— $r0$: Sommet de départ (par défaut 1)
Sorties :
— $pred$: Liste des prédécesseurs définissant l'arbre couvrant
Début :

1. Initialiser $GraphePrim$ avec n sommets
2. $pres \leftarrow$ Tableau de booléens initialisé à `Vrai` (indique si un sommet est dans le tas)
3. $GraphePrim.cle[r0] \leftarrow 0$
4. $GraphePrim.echanger(1, GraphePrim.noeudTas[r0])$ // *Placer r0 à la racine*
5. Tant que $GraphePrim.tas$ n'est pas vide faire :
6. $u \leftarrow GraphePrim.tas[1]$ // *Extraire le min*
7. $pres[u] \leftarrow$ `Faux`
8. $GraphePrim.echanger(1, \text{taille}(tas))$
9. Retirer le dernier élément de $GraphePrim.tas$
10. Si $GraphePrim.tas$ non vide alors :
11. $GraphePrim.versLeBas(1)$
12. Pour chaque sommet v de 1 à n faire :
13. Si $pres[v]$ et $D[u, v] < GraphePrim.cle[v]$ alors :
14. $GraphePrim.cle[v] \leftarrow D[u, v]$
15. $GraphePrim.pred[v] \leftarrow u$
16. $GraphePrim.versLeHaut(GraphePrim.noeudTas[v])$
17. Retourner $GraphePrim.pred$

Algorithme : PredToTree
Description : Convertit la liste des prédécesseurs en une structure d'arbre (dictionnaire d'adjacence).
Vars :
— d : Dictionnaire (Liste d'adjacence)
— $racine$: Entier (Indice de la racine)
— i : Entier (Indice de boucle)
Entrées :
— $pred$: Liste des prédécesseurs
Sorties : d (arbre), $racine$
Début :

1. $d \leftarrow$ Dictionnaire vide
2. $racine \leftarrow -1$
3. Pour i allant de 1 à taille$(pred)$ faire :
4. $\quad d[i] \leftarrow$ Liste vide
5. $\quad$ Si $pred[i]$ est `None` ou -1 alors :
6. $\quad\quad racine \leftarrow i$
7. Pour i allant de 1 à taille$(pred)$ faire :
8. $\quad$ Si $pred[i]$ n'est pas `None` et $pred[i] \neq -1$ alors :
9. $\quad\quad$ Ajouter i à $d[pred[i]]$
10. Retourner $d, racine$

Algorithme : ParcoursPrefixe
Description : Effectue un parcours en profondeur (DFS) pour obtenir l'ordre de visite des sommets.
Vars :
- $stack$: Pile d'entiers
- p : Tableau d'entiers (Marquage visite)
- p_etoile : Liste d'entiers (Résultat)
- $count$: Entier
- lst_voisin : Liste d'entiers
- rep : Booléen
- $voisin$: Entier

Entrées :
- d : Dictionnaire d'adjacence
- $racine$: Entier

Sorties : p_etoile (liste ordonnée)
Début :

1. $stack \leftarrow [racine]$
2. $p \leftarrow$ Tableau de zéros de taille n
3. $p_etoile \leftarrow [racine]$
4. $p[racine] \leftarrow 1$
5. $count \leftarrow 2$
6. Tant que $stack$ n'est pas vide faire :
7. $\quad lst_voisin \leftarrow d[\text{sommet}(stack)]$
8. $\quad rep \leftarrow$ Faux
9. $\quad$ Pour chaque $voisin$ dans lst_voisin faire :
10. $\quad\quad$ Si $p[voisin] == 0$ alors :
11. $\quad\quad\quad$ Empiler $voisin$ sur $stack$
12. $\quad\quad\quad$ Ajouter $voisin$ à p_etoile
13. $\quad\quad\quad p[voisin] \leftarrow count$
14. $\quad\quad\quad count \leftarrow count + 1$
15. $\quad\quad\quad rep \leftarrow$ Vrai
16. $\quad\quad\quad$ Sortir de la boucle Pour
17. $\quad$ Si non rep alors :
18. $\quad\quad$ Dépiler de $stack$
19. Retourner p_etoile

Algorithme : Procédure OptPrim
Description : Construit un cycle hamiltonien à partir de l'arbre couvrant minimal.
Vars :
- $pred$: Liste d'entiers
- d : Dictionnaire
- $racine$: Entier
- c : Liste d'entiers
- l : Réel

Entrées :
- $points$: Liste de points
- D : Matrice des distances
- $r0$: Sommet de départ

Sorties : c, l
Début :

1. Si D est `None` alors :
2. $\quad D \leftarrow$ `CalculerMatriceDistances`$(points)$
3. $pred \leftarrow$ `Prim`$(D, r0)$
4. $(d, racine) \leftarrow$ `PredToTree`$(pred)$
5. $c \leftarrow$ `ParcoursPrefixe`$(d, racine)$
6. $l \leftarrow$ `CalculerLongueur`(c, D)
7. Retourner c, l

Implémentation Python :

```python
class GraphePrim:
    def __init__(self, n):
        """
        Initialise la structure de données pour l'algorithme de Prim.
        Args:
            n (int): Nombre de sommets du graphe.
        """
        self.cle = [float('inf')] * n
        self.pred = [None] * n
        self.noeudTas = list(range(n))
        self.tas = list(range(n))

    def echanger(self, i, j):
        """
        Échange deux éléments dans le tas et met à jour leurs positions.
        Args:
            i (int): Indice du premier élément dans le tas.
            j (int): Indice du deuxième élément dans le tas.
        """
        temp = self.tas[i]
        self.tas[i] = self.tas[j]
        self.tas[j] = temp
        self.noeudTas[self.tas[i]] = i
        self.noeudTas[self.tas[j]] = j

    def priorite(self, i):
        """
```

```
        Retourne la priorité (clé) d'un élément à une position donnée
    dans le tas.
        Args:
            i (int): Indice dans le tas.
        Returns:
            float: La valeur de la clé associée.
        """
        return self.cle[self.tas[i]]

    def versLeHaut(self, i):
        """
        Fait remonter un élément dans le tas pour respecter la propriété
     de tas min.
        Args:
            i (int): Indice de l'élément à faire remonter.
        """
        while i > 0 and self.priorite(i) < self.priorite((i - 1) // 2):
            self.echanger(i, (i - 1) // 2)
            i = (i - 1) // 2

    def versLeBas(self, i):
        """
        Fait descendre un élément dans le tas pour respecter la propriét
    é de tas min.
        Args:
            i (int): Indice de l'élément à faire descendre.
        """
        s = 2 * i + 1
        while s < len(self.tas):
            if s + 1 < len(self.tas) and self.priorite(s + 1) < self.
    priorite(s):
                s += 1
            if self.priorite(i) > self.priorite(s):
                self.echanger(i, s)
                i = s
                s = 2 * i + 1
            else:
                break

def prim(D, r0=0):
    """
    Algorithme de Prim pour trouver l'arbre couvrant de poids minimal.
    Args:
        D (np.ndarray): Matrice des distances
        r0 (int, optional): Sommet de départ. Par défaut 0.
    Returns:
        list: Liste des prédécesseurs (pred[i] est le parent de i dans l
    'arbre)
    """
    n = len(D)
    graphe = GraphePrim(n)
    pres = [True] * n

    # Initialiser la clé du sommet de départ
    graphe.cle[r0] = 0
    # Placer r0 à la racine du tas
    graphe.echanger(0, graphe.noeudTas[r0])
```

```
    while len(graphe.tas) > 0:
        u = graphe.tas[0]
        pres[u] = False

        graphe.echanger(0, len(graphe.tas) - 1)
        graphe.tas.pop()

        if len(graphe.tas) > 0:
            graphe.versLeBas(0)

        for v in range(n):
            if pres[v] and D[u, v] < graphe.cle[v]:
                graphe.cle[v] = D[u, v]
                graphe.pred[v] = u
                graphe.versLeHaut(graphe.noeudTas[v])

    return graphe.pred

def pred_to_tree(pred):
    """
    Convertit la liste des prédécesseurs en une structure d'arbre (
    dictionnaire d'adjacence).
    Args:
        pred (list): Liste des prédécesseurs où pred[i] est le parent de
    i.
    Returns:
        tuple: (d, racine) où d est le dictionnaire d'adjacence et
    racine est l'indice de la racine.
    """
    d = {}
    racine = -1
    for i in range(len(pred)):
        d[i] = []
        if pred[i] is None or pred[i] == -1:
            racine = i

    for i in range(len(pred)):
        if pred[i] is not None and pred[i] != -1:
            d[pred[i]].append(i)

    return d, racine


def parcours_prefixe(d, racine):
    """
    Effectue un parcours préfixe (DFS) de l'arbre pour obtenir l'ordre
    de visite des sommets.
    Args:
        d (dict): Dictionnaire d'adjacence de l'arbre.
        racine (int): Indice de la racine de l'arbre.
    Returns:
        list: Liste des sommets dans l'ordre du parcours préfixe.
    """
    stack = [racine]
    p = [0] * len(d)
    p_etoile = [racine]
    p[racine] = 1
    count = 2
```

```
    while len(stack) > 0 :
        lst_voisin = d[stack[len(stack) -1]]
        rep = False

        for i in range(len(lst_voisin)):
            voisin = lst_voisin[i]
            if p[voisin] ==0 :
                stack.append(voisin)
                p_etoile.append(voisin)
                p[voisin] = count
                count +=1
                rep = True
                break

        if not rep :
            stack.pop()
    return p_etoile

def optPrim(D, r0=0):
    """
    Procédure OptPrim : construit un cycle hamiltonien à partir de l'
   arbre couvrant minimal en utilisant un parcours préfixe.
    Args:
        D (np.ndarray): Matrice des distances.
        r0 (int, optional): Sommet de départ pour Prim. Par défaut 0.
    Returns:
        tuple: (c, l) cycle hamiltonien (liste d'indices), longueur du
   cycle
    """
    pred = prim(D, r0)
    d, racine = pred_to_tree(pred)
    c = parcours_prefixe(d, racine)

    # Calcul de la longueur totale du cycle
    l = 0.0
    n = len(c)
    for k in range(n):
        l += D[c[k], c[(k + 1) % n]]

    return c, l
```

Listing 4 – Implémentation de OptPrim

2.4 OptPrim + Décroisé (OptOptPrim)

Algorithme : Procédure OptOptPrim
Description : Combine OptPrim et le décroisement.
Vars :
— c_{init} : Liste d'entiers (Cycle initial)
Entrées :
— $points$: Liste de points
— D : Matrice des distances
— $r0$: Sommet de départ
Sorties : c, l
Début :

1. Si D est `None` alors :
2. $D \leftarrow$ `CalculerMatriceDistances`$(points)$
3. $(c_{init}, _) \leftarrow$ `OptPrim`$(points, D, r0)$
4. Retourner `Décroise`(c_{init}, D)

Implémentation Python :

```python
def optOptPrim(D, r0=0):
    """
    Procédure OptOptPrim : combine OptPrim (MST + DFS) et décroisement.
    Génère un cycle initial avec l'heuristique de l'arbre couvrant
   minimum,
    puis l'améliore avec une recherche locale (décroisement).

    Args:
        D (np.ndarray): Matrice des distances.
        r0 (int, optional): Sommet de départ pour Prim. Par défaut 0.
    Returns:
        tuple: (c, l) cycle hamiltonien optimisé, longueur du cycle
    """
    # Étape 1: Obtenir le cycle initial via Prim + Parcours Préfixe
    c_init, _ = optPrim(D, r0)

    # Étape 2: Optimiser ce cycle avec décroisement
    return decroise(c_init, D)
```

Listing 5 – Implémentation de OptOptPrim

2.5 Évaluation et séparation progressive ("branch and bound") (HDS) pas fait

Section à compléter.